Implementation and Performance Analysis of Control Methods in an Embedded Smart Home Prototype

Sultan Olaogun

SID: 2228531

MOD002691 Final Project

Final Project Report

BEng Software Engineering

Submitted: April 2026

Abstract

The project presents the design and implementation of cost-effective home automation and smart lighting solutions that integrate cloud platforms, mobile control interfaces, and automated occupancy detection. The system enables real-time RGB LED manipulation through Android mobile applications with state synchronized via AWS cloud services including RDS MySQL databases, Lambda serverless functions, and API Gateway RESTful endpoints. An ESP-32 microcontroller polls cloud state at one-second intervals to drive WS2812B addressable LED strips while monitoring IR break beam sensors for directional occupancy tracking adapted from security system methodologies.

Development followed an iterative Agile methodology adapted to suit a solo developer with backend infrastructure prioritized as the foundational layer before simultaneous mobile and embedded implementation. Technology selection emphasized cost optimization and speed, with MySQL chosen for read-heavy performance, Python Lambda for extensive library support despite slower cold starts, and ESP-32 for integrated Wi-Fi with sufficient memory for HTTPS certificate validation. Testing utilized PowerShell for API validation, MySQL Workbench for database verification, and Serial Monitor for firmware debugging, enabling the discovery of failures before they became cascading failures.

The implemented system achieves hardware costs under £40 with monthly expenses being able to be mitigated through using AWS Free Tier and right sizing database instances. While commercial alternatives do provide greater functionality including more options for home automation, the primary functionality is comparable to that of commercial alternatives. The project demonstrates smart home automation development accessibility for those who are technically proficient, through a series of hardware, open-source software, and modern cloud platforms solutions.

Acknowledgements

Table of Contents

Abstract.....ii

Acknowledgements..... iii

Table of Contents..... iv

List of Figures.....vi

List of Tables..... vii

1.0 Introduction..... 1

1.1 Aims of the study..... 1

1.2 Background..... 2

2.0 Methodology..... 4

2.1 A description of Methodology..... 4

2.2 Specific Technology Selection..... 5

2.2.1 Database..... 5

2.2.2 Lambda..... 6

2.2.3 Microcontroller..... 7

2.2.4 User Interface..... 8

2.2.5 Lighting..... 8

2.3 Testing Methodology..... 9

2.4 Limitations..... 10

3.0 Design..... 11

3.1 Architecture Overview..... 11

3.2 Database…… 12

3.3 API…… 13

3.4 Lambda…… 14

3.5 ESP-32/Lighting…… 15

3.6 Android App…… 16

4.0 Implementation and Results…… 17

4.1 Backend Implementation…… 17

4.2 Database…… 19

4.3 Networking…… 20

4.4 Hardware Implementation…… 21

4.5 Android App Implementation…… 23

4.6 Testing Results…… 24

5.0 Discussion…… 26

5.1 Objectives…… 26

5.2 Technical Challenges…… 27

5.3 Learning Outcome…… 28

5.4 Future Improvements…… 29

6.0 Conclusion…… 30

6.1 Conclusion…… 30

References…… viii

Appendix I – Interim Report…… ix

Appendix II – Poster…… x

Appendix III – API Gateway Structure…… xi

List of Figures

Figure 4.1: Lambda function structure showing database helper integration……17

Figure 4.2: db.py helper file for standardized response formatting……18

Figure 4.3: MySQL Workbench database schema diagram……19

Figure 4.4: API Gateway resource hierarchy……20

Figure 4.5: API Gateway method execution for GET and POST……20

Figure 4.6: PowerShell API testing output……24

Figure 4.7: MySQL Workbench foreign key constraint validation……25

Figure 4.8: ESP-32 Serial Monitor latency testing results……25

List of Tables

Table 3.1: Comparison of in-game 3-marine resource building completion times……10

Table 3.2: Comparison of run time executions…… 12

1.0 Introduction

Modern smart lighting solutions primarily function on closed ecosystems, often require a non-insignificant financial investment. Traditional lighting solutions offer primarily binary on and off functionality with dimmable solutions requiring specialized bulbs, modified wall switches and are at times incompatible with home electrical systems. This solution is not accessible to a significant number of customers and lacks the ability to customize colours or schedule any form of automation. This project addresses these limitations by developing a cost-effective lighting system integrating cloud infrastructure, mobile control interfaces, and automated occupancy detection to provide commercial-grade functionality at a fraction of typical market costs. The system demonstrates how modern cloud services, embedded systems and mobile development can be combined to create practical home automation systems that are accessible to both developers and consumers.

1.1 Aims of the study

The aim of the study is to design and implement a full stack lighting control system that enables real-time remote controlled lighting manipulation. This project will:

- Develop a cloud-based infrastructure using AWS services to provide RESTful API endpoints for lighting management, motion sensing and occupancy tracking using HTTPS for secure communication.

- Implement an ESP-32 based embedded system with HTTPS certificate validation to poll cloud state, drive LED lights and detect room occupancy using IR break-beams.
- Create an android app that provides the user with an intuitive UI for colour selection, power control, motion sensing and occupancy display.
- Achieve sufficient functionality while maintaining lower cost than production grade solutions while maintaining comparable features.

1.2 Background

The home automation industry has seen significant growth in recent years. The widespread usage of technologies created for home automation has transformed IoT devices from a novel luxury to an accessible commodity, with global smart home markets expected to continue growing. Smart lighting represents a significant portion of the market, and this is driven primarily by a rise in consumer demand for energy efficiency, convenience and customization. Lighting is no longer merely how we make things brighter, hence the popularity of coloured LED lighting fixtures. The largest competitors in the market include Philips Hue, Nanoleaf and Govee. These companies have built feature rich ecosystems with a variety of high-quality products but they often exceed the average consumers budget for their starter packs. These systems also tend to employ proprietary tools and have closed ecosystems and require specific hubs and applications in order to configure products. These systems are not designed with ease of repair or compatibility with other ecosystems in mind.

Feedback from user experience typically favours home automation systems with open architectures and standard communication across services that address ecosystem

limitations. RESTful API design principles enable cross platform integration where clients are able to communicate with each other. One example of a company who aims to address these concerns is Meross, who have created public API that can be used to control and configure lighting. Cloud computing platforms like AWS and Microsoft Azure can provide serverless environments where users can also create and host their own cloud-based resources to create their own personal home automation systems that are capable of interacting with public API's. The serverless services can prove advantageous especially for home automation systems that see sporadic requests after long periods of rest as cloud providers offer resources with usage-based pricing where one could pay almost nothing monthly when provisioning resources for small scale projects like a single homes automation system.

2.0 Methodology

2.1 A description of Methodology

The project used an Agile methodology that was slightly modified to be able to be used by a solo developer. Components of the project were built iteratively and tested at the end of a components completion before being fully integrated into the project. The project was designed this way to ensure the multiple different technologies would work together in a real-world environment. Each system was designed to only interact with a select few other system in order to reduce complexity.

Initial prototyping focused on the backend infrastructure first as the foundational layer for the project. The database was designed and implemented first using MySQL and hosted using AWS Relational Database Service (RDS) and tested and accessed using SQL queries in MySQL Workbench. The API was then set up and configured to process GET and POST responses from users and set up to integrate AWS Lambda Python 3 functions to access data. I then designed and implemented the Lambda functions and continuously tested data retrieval and updates using Windows PowerShell. Once it was established the backend systems were working as expected and data could successfully be retrieved and updated, the next step became to move onto the user facing elements of the project

The ESP-32 code and wiring as well as the Android app were designed and implemented simultaneously as they both accessed the same backend system. I used Android Studio to develop the app and the Arduino Integrated Development Environment (IDE) to write and flash the firmware for the ESP-32.

2.2 Specific Technology Selection

2.2.1 Database

The choice to use MySQL was driven primarily by practical performance considerations and data structure requirements. The project requires data stored to be in a structured environment, with a one-to-one relationship and clear child/parent relationships. For this a traditional SQL database was believed to be the best decision. SQL also provided an advantage in terms of familiarity due to previous experience working with the language in previous projects, this would reduce development time required to complete this component of the project, as opposed to learning then configuring NoSQL databases such as DynamoDB. As mentioned previously, the backend was developed using AWS free tier which offers an additional $100 credit for activating an AWS RDS, which would effectively reduce the cost incurred to zero. MySQL was chosen specifically because of its performance in read heavy environments, and to polling the database in real time relies more heavily on read operations as opposed to writes.

2.2.2 Lambda

Lambda with Python 3.14 runtime was chosen as the API execution method because of the language characteristics and serverless execution model. While Lambda has no operations that it needs to execute the service remains idle until it is needed. With home automation systems having sporadic request patterns where no operations may be required for an extended period of time, and AWS's pay-as-you-go pricing model where cost is only incurred when a service is used, the need for a serverless solution becomes apparent. This solution is less expensive and uses less compute resources than a traditional always-on server solution. During the selection for the language that would

be used in Lambda, the main factor that shaped decision making was how long it would take to initially startup or cold start as due to the sporadic nature of home automation systems, there would likely be long gaps in between operations and this would cause the server to be slower on startup. A custom runtime or Node.js have the fastest cold starts but based on familiarity with different runtimes, and the length of time it would have taken to learn to use them, the decision was made to opt for runtime with the next fastest cold start, being Python. Python was also chosen because of its extensive repertoire of well-maintained libraries that would be needed to facilitate database connections and JSON parsing. While Python is slower than other languages after the cold start stage as Python is an interpreted language, the difference in latency is negligible and would not cause any significant latency issues on the back end.

2.2.3 Microcontroller

When deciding the method that would be used to access the cloud-based database and drive the LED lights, it was necessary to consider which solution would be the most practical. It was initially planned to use an Arduino R4 Wi-Fi microcontroller but limitations to accessing the boards built in ESP-32 chip made it necessary to make changes to the design. The ESP-32-S3 chip that was built into the board boasts a greater quantity of RAM than the Arduino R4 board itself that would be required for TLS, but libraries required to facilitate connection were designed to use the boards DRAM which is not sufficient for TLS as opposed to the slower but more voluminous PSRAM. The choice was then made to use a standalone ESP-32 WROVER which has sufficient DRAM for TLS. Using a Raspberry Pi Module was also considered because of its greater computational capabilities but this was deemed as excessive for the scope of the project as a Raspberry Pi would have a greater power consumption, and the operating system would only unnecessarily add to the complexity of the project.

2.2.4 User Interface

When deciding how the user would interact with the system, during the early stages of design and testing, the plan was to use an Infrared (IR) remote control and IR receiver to control the lights. This idea was abandoned as it lacked the ability to scale up the project and allow the user to control multiple lights with their device without building and designing a new custom IR remote controller to allow the user to control more than one set of lights, which would increase the cost of the project and the time required to complete. For this reason, there was a change made to pivot to designing a simple android application as part of the front end and user interface with Java working to interact with an API and give information about the lights current state to the user.

2.2.4 Lighting

The lights used in the project were WS2812B Addressable RGB (ARGB) lights with 144 individual lights. The plan was to use a set number of individual lights to represent individual rooms in a house.

2.3 Testing Methodology

Testing was done along side integration of components to examine how the system behaved in specific circumstances. AWS CloudWatch was used to give additional information when components would not act as anticipated and gave valuable information that aided in fixing errors and improving performance. PowerShell's Invoke-RestMethod cmdlet served as the primary API testing tool during development and provided access to send HTTPS requests directly to the API without the need for a user interface during earlier stages of development. It allowed for the sending of JSON payloads and would also allow for logging of responses from the API. This approach

enabled sequential implementation of new methods and testing without requiring other components such as the Android application or the ESP-32.

Hardware testing began with smaller testing to isolate key issues. Initial LED testing used a smaller set of LED lights to verify data integrity and ensure the correct colours and number of LED's could be displayed before implementing multiple room control. This incremental approach allowed for the discovery of power inadequacies and data driving related issues, which were later fixed.

Full integration testing with multiple rooms being controlled by the users device via the Android application to change light colours in real time, database state verification and physical LED appearance was done using the Serial Monitor in the Arduino IDE to provide JSON payloads that had been converted to RGB/HSV values to verify whether the correct data had been successfully extracted from the database.

2.4 Limitations

The method by which room occupancy was logged requires user disciple in practice. Should a user unintentionally trigger an event where a person has been erroneously detected to have left or arrived in the room, it could cause unintended changes in the physical LED's state. For this reason, an option to manually change the number of people currently in a room was added to the Android application. Another limitation is that the system is designed to detect the number of people who enter or leave a room and it is unable to accurately count the actual number of people in a specific room.

Power requirements were not conducted quantitatively prior to implementation, and the solution had to be changed at multiple points throughout the lifecycle of the project due to this oversight. Power consumption was also not accounted for which caused

issues during development as the solution had to be changed from a battery powered solution to a dedicated power supply in order to facilitate long term operation of both LED’s and microcontroller.

3.0 Design

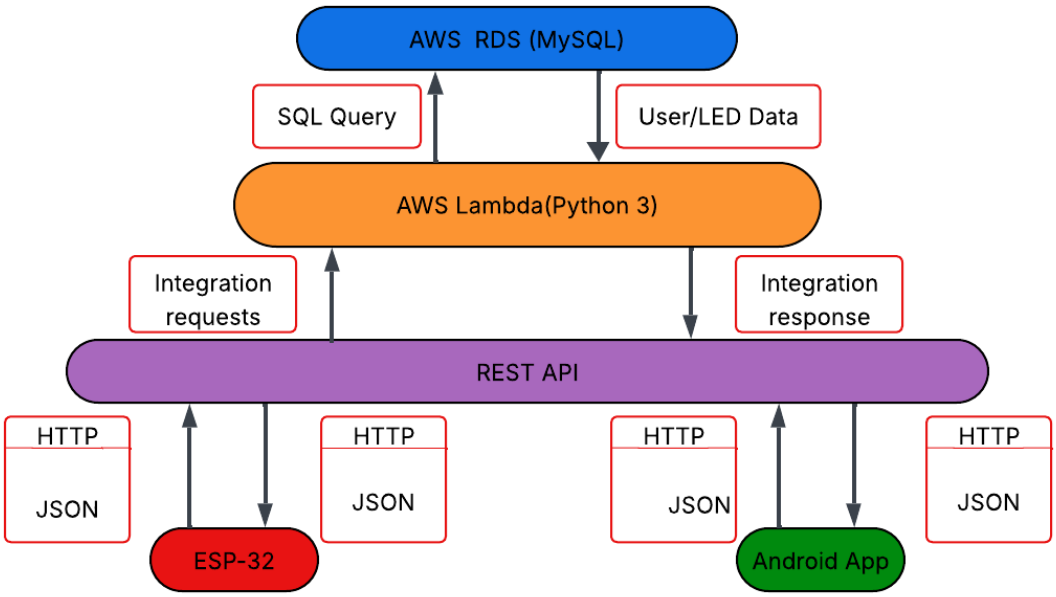
3.1 Architecture Overview

The system architecture implements a 3-tiered model, implementing user presentation, application logic and data storage. The presentation tier consists of the Android application that provides the user with an interface for controlling the lights and adjusting settings. The application utilises API gateway access for HTTPS endpoint exposure and AWS Lambda functions for logic executions on database hosted on AWS RDS.

Communication flows follow a response and request pattern with Android application and ESP-32 acting as the client the user interacts with the API. User actions in the application trigger POST requests to update the database data while the ESP-32 microcontroller sends GET requests at set intervals to update the physical LED lights on the user end. This architecture decouples the application from the microcontroller and handles any erroneous data. This allows for a user to be able to control more than one light source and any other microcontrollers to be able to be controlled using different devices.

The decision to have ESP-32 polling as opposed to data being pushed by the server to the microcontroller on update from the application component simplified the architecture significantly. While a push-on-update system would reduce latency and require fewer requests to be made and reduce server load, it requires more complex architecture and server-side management of resources. A polling-based architecture would be more tolerant of network issues such as temporary outages and can continue

operations more seamlessly as GET requests can simply be made in the next polling cycle and retrieve data without regard for missed intervals.



3.2 Database

The database consists of 4 tables to store data for rooms and lights. Those tables being rooms, led_state, motion_settings and occupancy. The rooms table serves as the central entity with the room_id serving as the primary key, the name field for a human readable representation of the individual room and a created_at timestamp for potential future integrations

The led_state table serves as the cloud-based state of the lights in a particular room. It contains room_id from rooms as the foreign key and light_id as the primary key for the database. It also contains the values for the lights, those values being: is_on for allowing the user to turn the lights on or off, hue, saturation and value for deciding the colour and intensity of the lights. The purpose of the last_updated timestamp is to help log changes for debugging and analytics.

The motion_setting and occupancy follow a similar and simple pattern. They both include room_id from rooms as a foreign key as well as their own respective id as their primary key. The motion_settings table includes a column named motion_sensing_enabled which determines whether or not motion sensing will be active and occupancy holds a column named people_count to store the number of people that are present in a room. Both tables also hold a last_updated column included for later development as well as debugging and analytics.

3.3 API

The API is comprised of 8 different endpoints with a GET and POST endpoint for each method within the API. The endpoints have a hierarchical structure with rooms being at the top of said hierarchy. Rooms contains endpoints for /rooms/{id}/state, /rooms/{id}/motion and /rooms/{id}/occupancy. The API structure mimics the database structure in the sense the rooms table is the central table, and the rooms endpoint is the parent for all the other endpoints. All other endpoints are dependant on rooms and map Lambda functions to their respective tables in the database.

The API uses HTTPS methods and JSON payloads as the data format it uses as the mechanism for transferring data between different endpoints. The payloads contain variables related to database operations. GET methods are responsible for retrieving data from the database such as GET /rooms/{id}/state being used to retrieve the state of LED lights in the form of hue, saturation and value.

Post requests update LED state using JSON payloads containing corresponding value fields. POST /rooms/{id}/state contain the key-value pairs hue, saturation, value and is_on, POST /rooms/{id}/motion contains the key value pair

motion_sensing_enabled and POST /rooms/{id}/occupancy contains the key-value pair people_count.

All endpoints return JSON responses with the appropriate HTTP status codes, 200 for success, 404 when subject could not be found, 400 for bad requests and 500 when an error occurs on the server side. The responses are handled by serverless Lambda functions.

3.4 Lambda

The lambda functions server 2 primary functions, extracting data from JSON payloads to make changes to the database and giving responses back to the user to report successful or unsuccessful operations. There are 4 different Lambda functions, each responsible for one API endpoint and table in the database. Each function operates independently of other functions. This follows microservice architecture principles as the functions are loosely coupled and demonstrate the Single Responsibility Principle as each function is only responsible for its respective table. The functions also have a modular design and can be added, removed or updated without affecting overall functionality. The Lambda functions utilise the PyMySQL library to facilitate database connections to the cloud-based database.

The functions folder also includes a db.py file and PyMySQL folder as Lambda is not configured to import additional libraries unless they are included in the functions folder directory. These files are included in all Lambda functions. All functions using the same db.py file follows the DRY(Don’t Repeat Yourself) principle and establishes a common module pattern among functions. This approach ensures consistent database connections and simplifies error handling.

3.5 ESP-32/Lighting

The ESP-32 uses 2 functions; the setup() function and the loop() function, to complete various operations. The setup function initialises variables and sets up connection to the internet before attempting to drive the LEDs. The code for initialising the Wi-Fi connection is stored outside of the main file using a header and C++ file using the name Connectivity. The Connectivity file utilises the Wi-Fi library to facilitate the connection to local Wi-Fi networks. LED lighting also needs to be initialised before the loop() function begins. This is done by defining the variables required for the FastLED library to work and drive the LEDs.

During the loop, the ESP-32 sends GET requests to the API and receives HTTP responses with JSON payloads that contain the data required to change the colour and brightness of the LEDs. The ESP-32 frequently polls the API to ensure the lights are in sync with the state of the cloud-based database.

The ESP-32 and LED lights both require 5V of power to function correctly. The ESP-32 and lights will be connected to a breadboard to power both of the electronic components. A 330Ω resistor is also used to improve the integrity of data transferred from the ESP-32 to the addressable LED lights. A USB cable is capable of providing the 5V of power necessary to power the LEDs and the microcontroller.

3.6 Android App

The android app uses an Activity based architecture to display the lights the user has access to and allow them to control said lights. The main activity features a recycler view populated with rooms that exist within the cloud-based database using the GET

method to extract the rooms and their names. Each room can be accessed by clicking o them and will take the user to a page that will allow them to control a specific set of lights. Clicking on a specific room passed the room_name and room_id as Intent to identify which room should be controlled and that information is then passed onto the API to enable control.

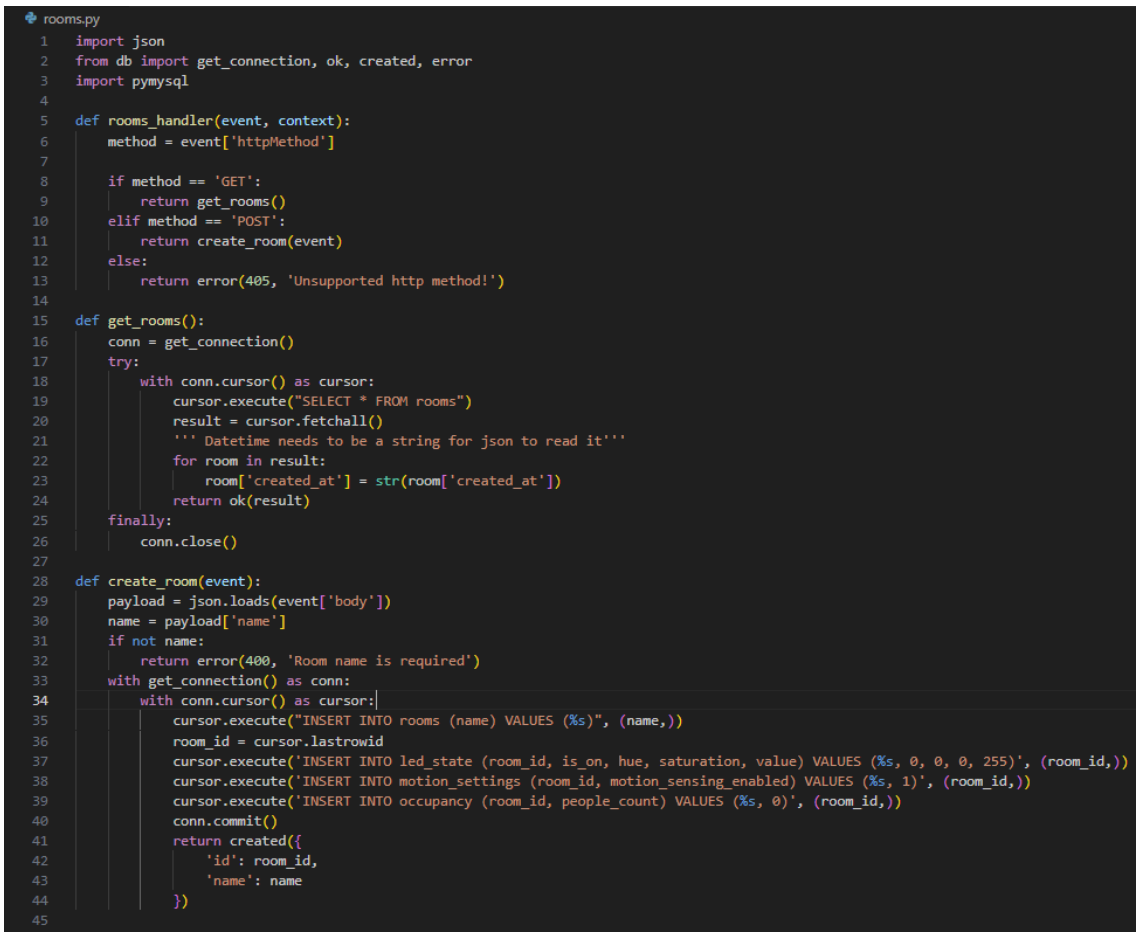
The activity that controls the room features a colour wheel / colour picker interface, power switch, update button, a toggle for setting the motion sensing and the estimated count of people in the room. The colour wheel uses a GitHub library Dhaval2404/ColorPicker to provide the user interface for colour selection as well as converting selected position on the colour wheel to HSV. The colour preview used to display the selected colour of the lights is converted to RGB to provide visual feedback to the user.

For communicating with the API, it was necessary to use AsyncTask despite it being deprecated. AsyncTask is used to run processes in the background of an application and prevent processes from slowing down the User Interface (UI). The updateLedState() method first calls this method and then creates a JSON payload from the current HSV values, on state, occupancy and motion settings. A HTTPS POST request is then sent in the background. This separates the UI from the network operations, thus preventing the UI from blocking network operations and vice versa.

4.0 Implementation and Results

4.1 Backend Implementation

The backend was implemented using a series of AWS Lambda functions written in Python 3.14. Each function was developed and tested once API was integrated. The testing used Microsoft PowerShell to validate changes made to the database by Lambda functions.



rooms.py

```python
import json
from db import get_connection, ok, created, error
import pymysql

def rooms_handler(event, context):
    method = event['httpMethod']

    if method == 'GET':
        return get_rooms()
    elif method == 'POST':
        return create_room(event)
    else:
        return error(405, 'Unsupported http method!')

def get_rooms():
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            cursor.execute("SELECT * FROM rooms")
            result = cursor.fetchall()
            ''' Datetime needs to be a string for json to read it'''
            for room in result:
                room['created_at'] = str(room['created_at'])
            return ok(result)
    finally:
        conn.close()

def create_room(event):
    payload = json.loads(event['body'])
    name = payload['name']
    if not name:
        return error(400, 'Room name is required')
    with get_connection() as conn:
        with conn.cursor() as cursor:
            cursor.execute("INSERT INTO rooms (name) VALUES (%s)", (name,))
            room_id = cursor.lastrowid
            cursor.execute('INSERT INTO led_state (room_id, is_on, hue, saturation, value) VALUES (%s, 0, 0, 0, 255)', (room_id,))
            cursor.execute('INSERT INTO motion_settings (room_id, motion_sensing_enabled) VALUES (%s, 1)', (room_id,))
            cursor.execute('INSERT INTO occupancy (room_id, people_count) VALUES (%s, 0)', (room_id,))
            conn.commit()
            return created({
                'id': room_id,
                'name': name
            })

```

led_state.py

```python
import json
from db import get_connection, ok, error

def handler(event, context):
    method = event['httpMethod']
    room_id = event['pathParameters']['id']

    if not room_id:
        return error(400, 'Missing room id')
    if method == 'GET':
        return get_led_state(room_id)
    elif method == 'POST':
        return update_led_state(room_id,json.loads(event['body']))
    else:
        return error(405, 'Method not allowed')

def get_led_state(room_id):
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            cursor.execute('SELECT is_on, hue, saturation, value FROM led_state WHERE room_id = %s', (room_id,))
            result = cursor.fetchone()
            if not result:
                return error(404, 'Room not found')
            result['room_id'] = int(room_id)
            result['is_on'] = bool(result['is_on'])
            result['hue'] = int(result['hue'])
            result['saturation'] = int(result['saturation'])
            result['value'] = int(result['value'])
            return ok(result)
    finally:
        conn.close()

def update_led_state(room_id, data):
    if 'hue' in data and not(0 <= data['hue'] <= 360):
        return error(400, 'Hue must be between 0 and 360')
    if 'saturation' in data and not(0 <= data['saturation'] <= 255):
        return error(400, 'Saturation must be between 0 and 100')
    if 'value' in data and not(0 <= data['value'] <= 255):
        return error(400, 'Value must be between 0 and 100')
    if 'is_on' in data and not isinstance(data['is_on'], bool):
        return error(400, 'is_on must 1 or 0')

    allowed = ['is_on', 'hue', 'saturation', 'value']
    fields = {k: v for k, v in data.items() if k in allowed}
    if not fields:
        return error(400, 'No valid fields')

    set_clause = ', '.join([f'{k} = %s' for k in fields.keys()])
    values = list(fields.values()) + [room_id]
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            cursor.execute(f'UPDATE led_state SET {set_clause} WHERE room_id = %s', values)
            conn.commit()
            return ok({'message': 'ok'})
    except Exception as e:
        print(e)
        return error(500, 'Internal server error')
    finally:
        conn.close()
```

The Lambda functions all follow a similar pattern where they receive a trigger from the API Gateway, extract parameters from request, validate inputs if required execute database operations and return JSON responses. Database connections are established using a database helper file and PyMySQL included in the same folder as the Lamda

function. The database connections are wrapped in try blocks to log errors and return the appropriate HTTP codes to clients.

db.py

```python
import os
import pymysql
import json

def get_connection():
    return pymysql.connect(
        host=os.environ("DB_HOST"),
        user=os.environ("DB_USERNAME"),
        password=os.environ("DB_PASSWORD"),
        database=os.environ("DB_NAME"),
        cursorclass=pymysql.cursors.DictCursor,
    )

def response(status_code, body):
    return {
        "statusCode": status_code,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Content-Type": "application/json",
            "Access-Control-Allow-Headers": "Content-Type",
            "Access-Control-Allow-Methods": "OPTIONS,POST,GET"
        },
        "body": json.dumps(body)
    }

def ok(body):
    return response(200, body)

def created(body):
    return response(201, body)

def error(status_code, message):
    return response(status_code, {'error': message})
```

Database connection management implements a simple approach where connection is initiated at the start of each request. Connections are implemented using the db.py file shown in the image above. This file connects to the database using environment variables defined in AWS Lambda's IDE and uses a dictionary cursor for the cursor

class as it was found to be the most readable and easily understood. The cursor is defined by the function that is importing db.py and the connection is established. An error response function is called depending on whether or not the try block executed successfully which then calls the response() function in db.py. Response is only included in db.py so the same lines of code do not have to be repeated in every function. This standardises the inclusion of headers, content type and status code among functions allowing for consistent responses and reducing the occurrence of unexpected responses.

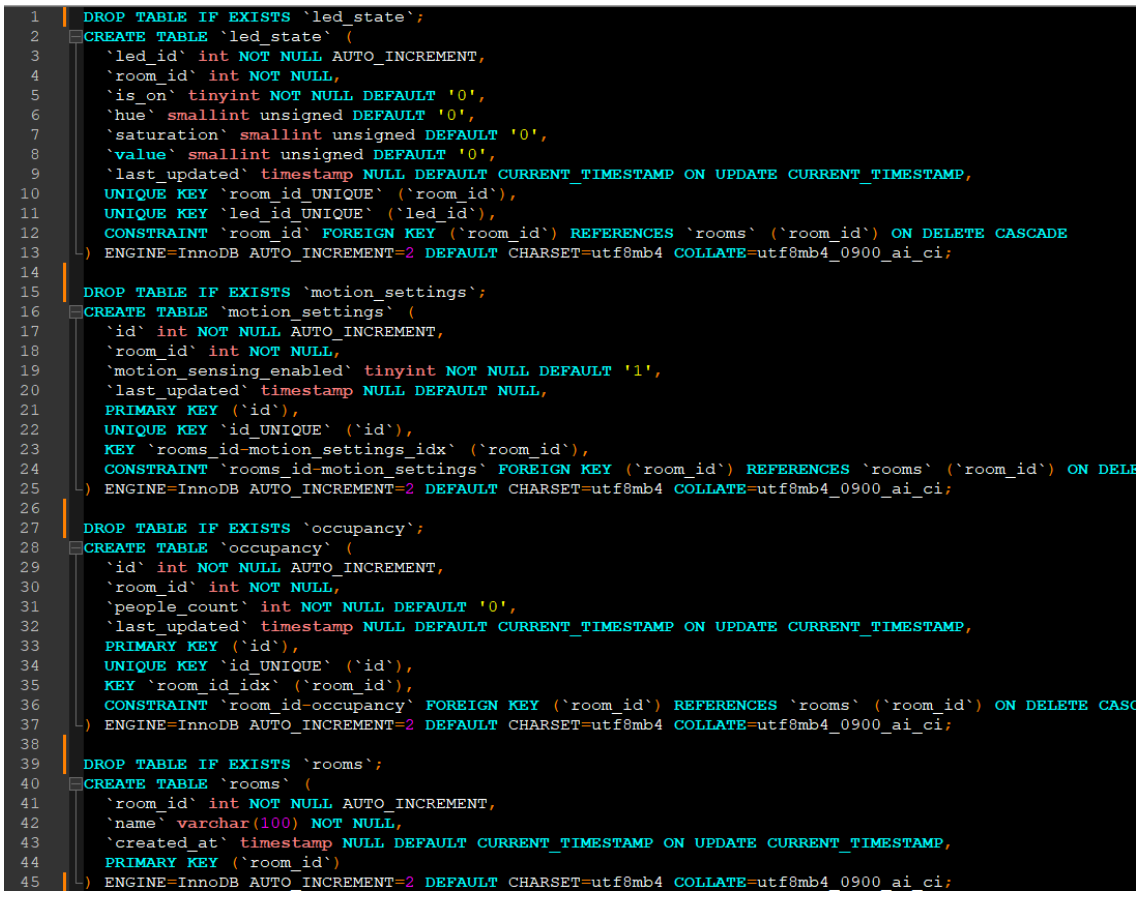
Lambda functions also made use of dynamic queries with an allowed list of parameters used to determine which fields are allowed to be edited by Lambda. The list of allowed parameters can also be changed at any time, allowing for future updates and developments to be made with ease. This approach also helps prevent SQL injection attacks by ensuring only predefined fields can be included in update statements and rejecting any malicious inputs.

During development an error occurred when attempting to access the API gateway from Android Studio. Whenever a GET request was sent to the API, the request would fail before getting to the API. Due to testing using Microsoft PowerShell Invoke-RestMethod, the cause of the error was quickly determined to be a malformed URL. Without the usage of this tool to directly interact with the API, testing and error handling would have been significantly more difficult and time consuming.

4.2 Database

The database implementation was very simple, I already had the list of tables and columns that were needed to create the database and all that was needed was to create the database using AWS RDS and edit using MySQL Workbench. MySQL Workbench offers a visual interface for designing database structures which was used to create this

database. The database was tested and had test queries executed on it prior to the creation of Lambda functions to make neccesary changes to the database.



```
DROP TABLE IF EXISTS `led_state`;
CREATE TABLE `led_state` (
  `led_id` int NOT NULL AUTO_INCREMENT,
  `room_id` int NOT NULL,
  `is_on` tinyint NOT NULL DEFAULT '0',
  `hue` smallint unsigned DEFAULT '0',
  `saturation` smallint unsigned DEFAULT '0',
  `value` smallint unsigned DEFAULT '0',
  `last_updated` timestamp NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY `room_id_UNIQUE` (`room_id`),
  UNIQUE KEY `led_id_UNIQUE` (`led_id`),
  CONSTRAINT `room_id` FOREIGN KEY (`room_id`) REFERENCES `rooms` (`room_id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

DROP TABLE IF EXISTS `motion_settings`;
CREATE TABLE `motion_settings` (
  `id` int NOT NULL AUTO_INCREMENT,
  `room_id` int NOT NULL,
  `motion_sensing_enabled` tinyint NOT NULL DEFAULT '1',
  `last_updated` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `id_UNIQUE` (`id`),
  KEY `rooms_id-motion_settings_idx` (`room_id`),
  CONSTRAINT `rooms_id-motion_settings` FOREIGN KEY (`room_id`) REFERENCES `rooms` (`room_id`) ON DELE
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

DROP TABLE IF EXISTS `occupancy`;
CREATE TABLE `occupancy` (
  `id` int NOT NULL AUTO_INCREMENT,
  `room_id` int NOT NULL,
  `people_count` int NOT NULL DEFAULT '0',
  `last_updated` timestamp NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `id_UNIQUE` (`id`),
  KEY `room_id_idx` (`room_id`),
  CONSTRAINT `room_id-occupancy` FOREIGN KEY (`room_id`) REFERENCES `rooms` (`room_id`) ON DELETE CASC
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

DROP TABLE IF EXISTS `rooms`;
CREATE TABLE `rooms` (
  `room_id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) NOT NULL,
  `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`room_id`)
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

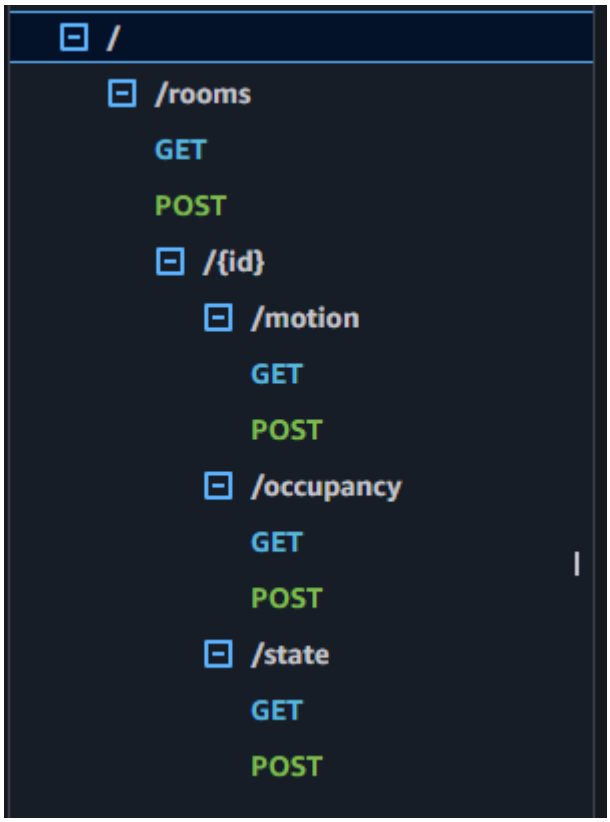
The above image is a cleaned up and readable schema produced by MySQL Workbench that demonstrates the SQL query required to replicate the structure of the database. Even though there is cuuently no method for deleting an room in the database, the ON DELETE CASCADE constarint is still required to ensure data integrity and that no other records exist that reference rooms that no longer exist.

- Another important aspect of creating the database was ensuring it was sufficiently normalised with shouldn't present much of a challenge with a databse of this size. It was decided the 3rd normal form was sufficient for this database. 1st Normal Form : The database meets the requirements as there are no repeated entries in the database.
- 2nd Normal Form : All key attributes are fully dependent on the primary key.

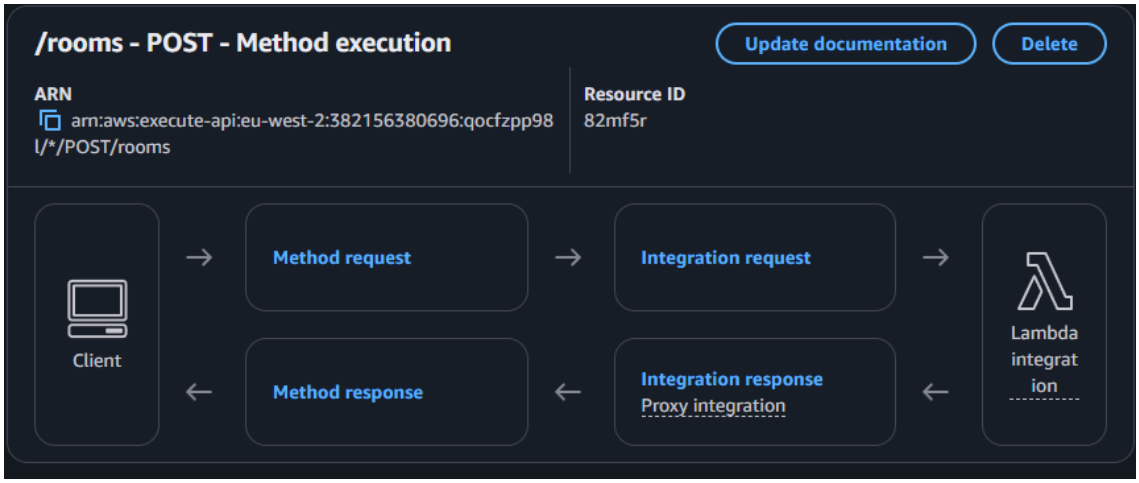
- 3rd Normal Form : All non-key attributes are dependant only on the primary key

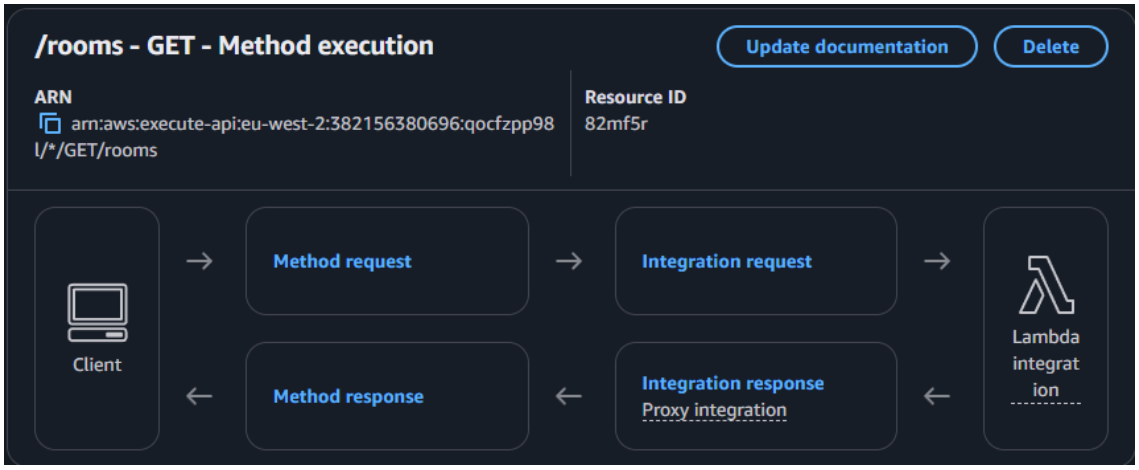
Database Cloud stuff. VPC, IGW NAT. During the creation of the database using AWS RDS it was neccesary to configure

4.3 API



The structure of the API does not deviate from the design outlined in the design section. It features the same hierarchical structure as previously mentioned, with {id} as the proxy resource that is used to determine which room is being accessed.





The two images above display the execution method for GET and POST methods on the /rooms resource. The method execution is the same for all resources in the API. Each resource is capable of sending integration requests to AWS services and then receiving responses from AWS services. In all resources the service being used is Lambda.

4.4 Networking

The during testing and implementation of the database, it was placed temporarily in a public Subnet Group inside of a Virtual Private Cloud (VPC). After implementing and testing it became necessary to place the database inside of a private subnet for security and to prevent it from being accessed by unauthorized third parties. The database settings were altered to disable public access, then using the dig command in Cloud

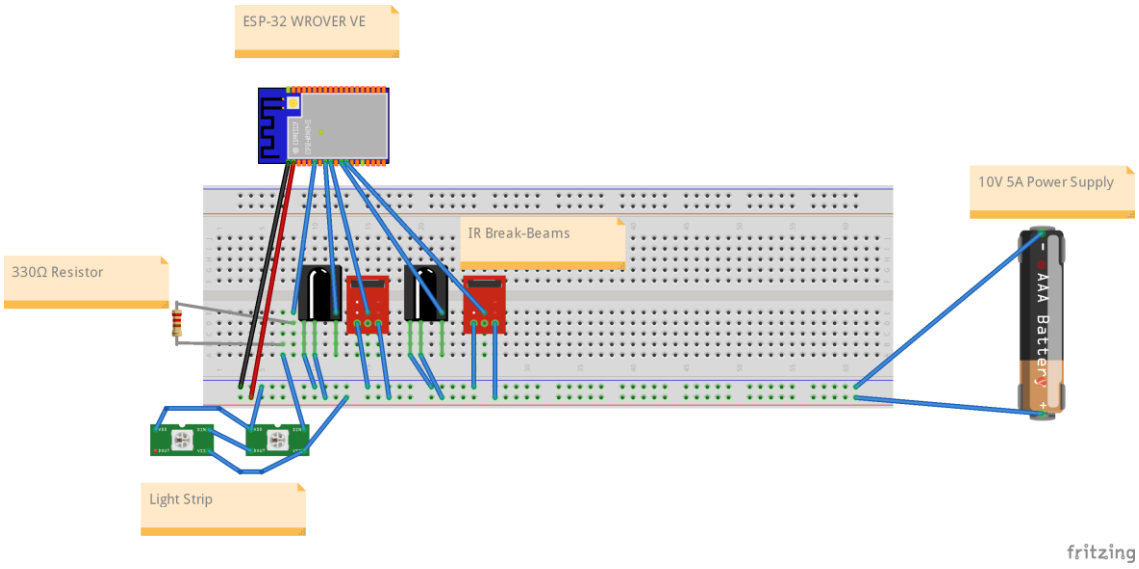
Shell the databases IP address could be found, then any subnet not being used could be removed from the Subnet Group. Multi-AZ deployment was then turned on to force the database to also use a private subnet; the private subnet was then made the primary instance and the secondary instance hosted on the public subnet could be removed from the Subnet Group.

The Lambda functions are also placed within the same VPC to allow it to communicate with the database without exposing it to public resources. Lambda functions also exist inside of private subnets to only allow it to be invoked by services that are manually connected to Lambda functions.

4.5 Hardware Implementation

The ESP-32 hardware implementation required careful attention to not just the firmware but also power management, as other electronic devices also needed to be managed such as the WS2812B ARGB strip or IR break-beam sensors. Initial testing using USB power revealed limitations that were not previously considered. While the USB power does meet the required Voltage(V) it was unable to provide sufficient amperage and was only able to power the first 30 lights, although this number was inconsistent. Amperage estimations for each LED on the strip were 60 milliamperes(mA) per LED and on a strip of 144, this would likely equal a total amperage of 8.64 amperes (A). This prompted a pivot to a dedicated power supply that was capable of delivering the necessary power to all of the electronics on the board. Although even with a new 5V 10A power supply, there was a noticeable dimming after 72 LEDs, so the decision was made to only use the 1st half of the light strip for

consistency.



A 330 Ohm (Ω) resistor was added to the data line connected to the light strip. This is done to dampen the signal and prevent signal reflections within the wire, improving signal integrity and preventing damage from power surges. A resistor was not required for the data lines on the IR break-beams because they have 2 states, either LOW or a floating state that can be read as HIGH but is unpredictable. To make this more reliable, we can use the internal pull-up resistors of the ESP-32. We do this by setting the pinMode of the specific pins that are being used to INPUT_PULLUP in the firmware.

The ESP-32 firmware implements the standard Arduino framework structure with setup() and loop() functions. Setup() executes one time on boot to set up Wi-Fi connections and initialise variables. The main loop() function executes continuously and handles API request, LED driving and occupancy tracking. The loop has a counter that is added to at the start of each loop, the purpose of this counter is to create delays between GET requests while also allowing occupancy to be detected. The way it works is at the beginning of the loop the counter is incremented by one, the break-beams are checked, if the loop counter is high enough, then the API GET requests are sent and the lights state is updated. The break-beam checks are done very quickly, there is a 50ms

delay at the end of the loop and the counter threshold for API requests to be sent is 20. This effectively makes the program send API requests every second and works to keep the lights updated in real time. The ESP-32 also utilizes TLS certificate validation to ensure secure communication. This was done using the WiFiClientSecure library and embedding the AWS Root CA certificate directly into the firmware.

The occupancy tracking implements aspects of security system implementations documented by Jose and Malekian (2017) where 2 sensors are used to detect the arrival of a person and the opening of a door. Sensor 1 being positioned in front of the door outside of the doors swing radius and Sensor 2 being positioned over the door inside the swing radius of the door. Sensor 1 detecting change and then sensor 2 detecting change not long after would indicate a person walked in front of the door, opened it and walked out. The opposite detection events would indicate a person opened the door and then stepped into the room. Depending on the order of detection events and the amount of time that had passed between events, the estimated number of people in a room can be calculated, given the initial number of people listen in the room is correct.

LED controller logic is dependent on a set of conditions. When is_on is returns true and either motion_sensing_enabled is disabled or the number of people in the rooms is greater than zero. The FastLED library expects values in the range of 0-255, this poses an issue in regard to hue which can range from 0-360. Thus, hue needs to be scaled down in order to be used to drive LED lights. To do this the formula “int hue_255 = (int)((hue/360.0) * 255);” needs to be applied. This converts hue to a percentage of 360 and multiplies it by 255 to return an integer in the acceptable range.

4.6 Android App Implementation

The android app uses an activity-based structure to provide the user with an interface they can use to control their lights. There are 2 main screen the user can interact with. The first is the home screen and contains a recycler view with rooms that are taken from the database. The 2nd activity is the page that is used to make changes to the lights such as colour and motion settings. This separation of ensures room specific controls and prevents instructions from being incorrectly sent to other rooms.

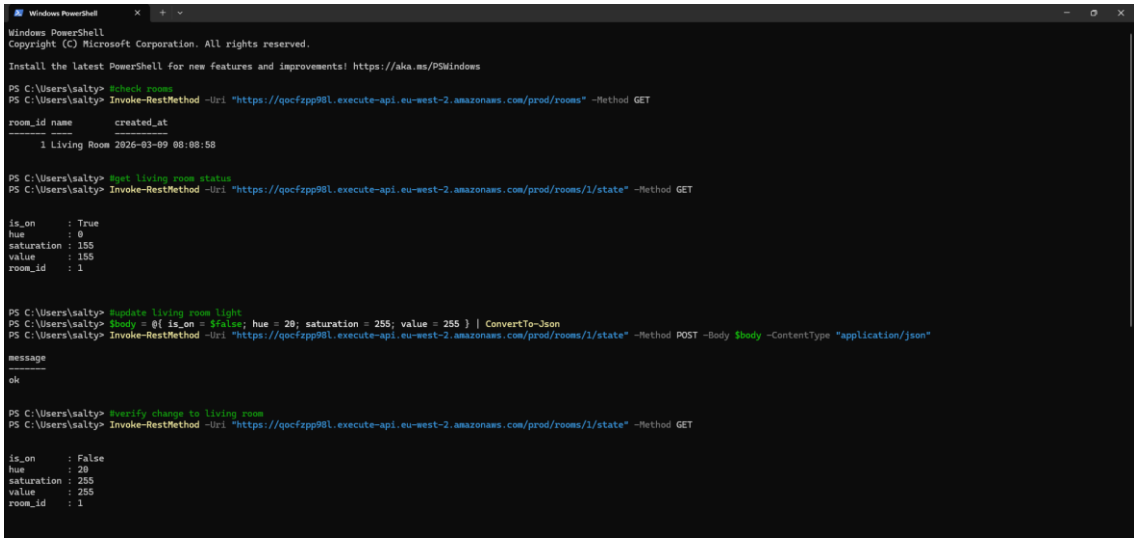
Communication with the API requires synchronous updates to provide immediate visual feedback to users. Executing API requests on the main UI thread is likely to cause the UI thread to buffer/freeze and cause delays when sending request. The app utilises AsyncTask despite its deprecation in newer android versions. Each API function implements AsyncTask to execute request in the background and processes responses using onPostExecute() on the main thread. This ensures that UI elements do not lag behind and are updated continuously while any network operations are handled seamlessly.

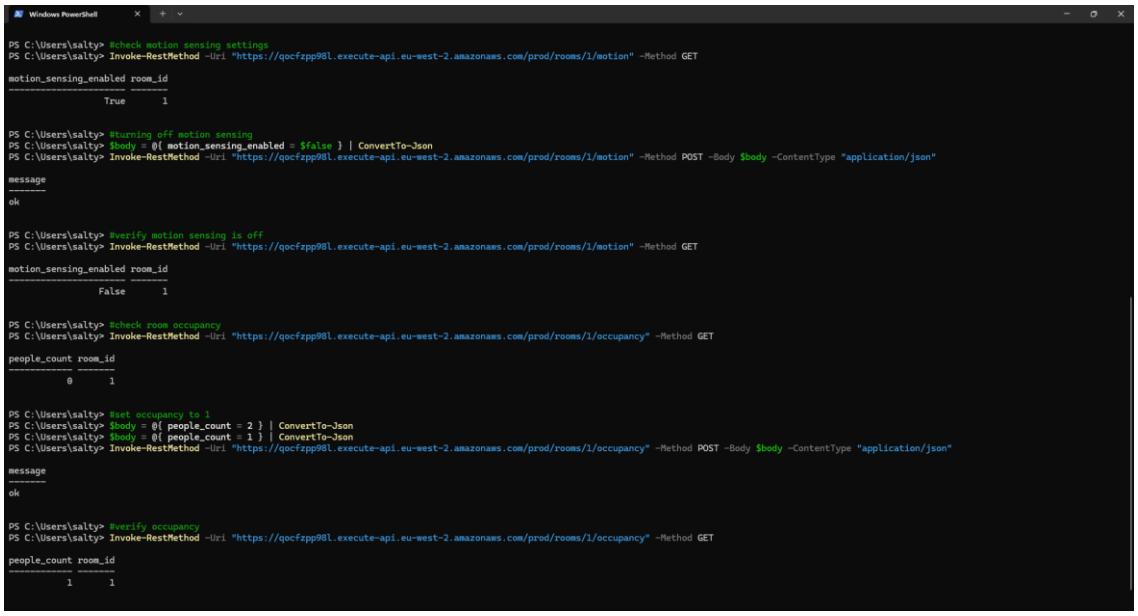
The application uses local state variables for hue, saturation, value, is_on, motion sensing and occupancy count. Data flows bidirectionally, with data associated with the rooms state and motion sensing being communicated to the database and the occupancy of the room being communicated to the app by the database. The app is primarily responsible for making changes to data in these fields and as a result are required to send frequent POST requests in relation to said fields. When the app starts up, it receives sends GET requests for the list of rooms that are available.

4.7 Testing Results

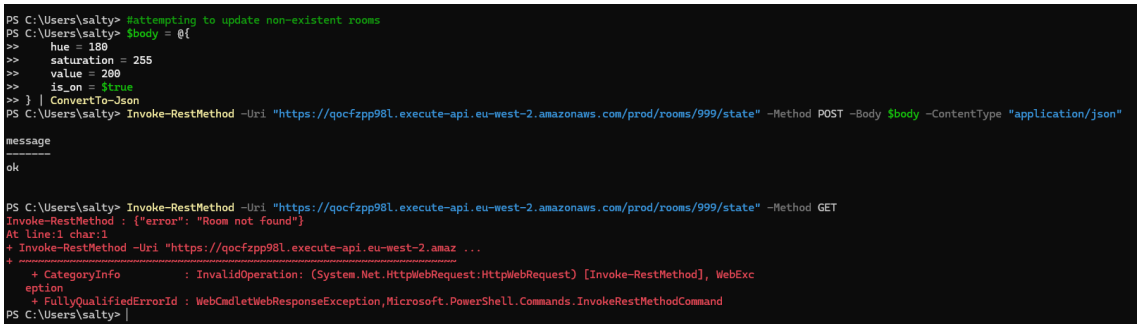
Testing was conducted incrementally alongside implementation to validate each component before fully integrating it. The incremental testing approach helped to identify issues that could have caused cascading failures across the system.

Microsoft PowerShell served as the primary testing tool for API validation during development. Each Lambda function was tested independently by using JSON payloads to verify functions work as expected and validate data entries. JSON payloads were sent to each endpoint in order to validate the systems function. This approach allowed for testing without the use of frontend implementation which allowed sufficient tests to be performed before the development of other components.





While the database was publicly accessible, MySQL Workbench provided direct database access for validating key Lambda functions and ensure data integrity. Foreign key relationships were tested by attempting to update room states with invalid room_id. The database's structure rejected the entry

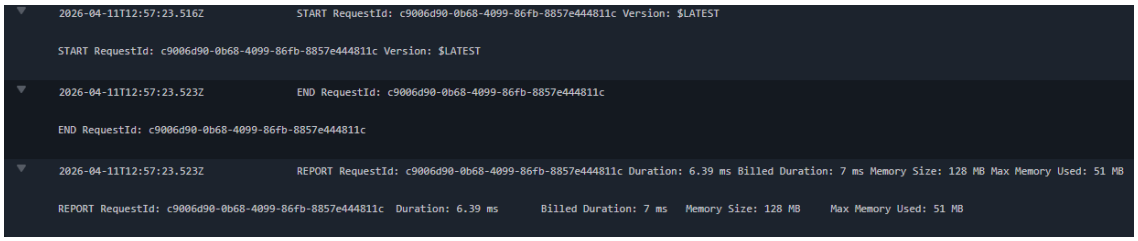


After the method was executed, the entry into the database could not be found. This indicates that the database itself rejected the entry even if Lambda allowed the request to go through.

Initial testing of the ESP-32 wasn't capable of testing on the full strip of LEDs due to power issues. During testing it was found that lighting intensity and colour would frequently fluctuate, thus a 330 Ohm resistor was added to the data line of the addressable LED lights to reduce the corruption that was occurring when sending data to lights. Serial Monitor output offered real time debugging information including Wi-Fi connection status, HTTP responses, HSV to RGB conversions and break-beam sensor validation. Break-beam sensor validation involved manually triggering each sensor as opposed to manually setting up sensors on a door to confirm detections.

Latency testing was also performed on the system. Android's HTTP requests were the main factor increasing latency as requests would typically take 200 milliseconds (ms) from connection opening to response receival. Lambda's latency differed at points throughout testing. When the functions had not been in use for prolonged periods of time, they could take around 150ms to complete operations but after they had been

warmed up those times dropped significantly, with response times below 10ms and database updates typically occurring within that 10ms timespan.



The ESP-32 has a polling rate of approximately 1 request per second. Reducing the polling rate would increase performance but will increase server-side costs as more requests will need to be made and the improvement will not be significant enough to justify the cost.

Overall, API response times averaged at around 200-300 ms when Lambda instances were warmed up and could take up to 1000ms for cold starts which only occur after being idle for extended periods of time. 1 second polling frequencies proved to be sufficient for home automation systems and were synchronized with the cloud-based database.

5.0 Discussion

5.1 Objectives

The project successfully delivered on providing users with core functionality, enabling the usage of mobile device controlled RBG customisable lighting through the usage of cloud configurations and microcontrollers, though some objectives remain partially achieved or require greater refinement at a later stage. The mobile application provides users with an intuitive user interface for colour selection and updating lighting fixtures and the microcontroller reliably drives the LEDs with minimal latency, however the system has limitations preventing it from reaching a similar level of quality, polish or reliability as commercial ecosystems. In an attempt to reduce cost incurred, a cheaper LED strip was employed which suffers from reduced colour accuracy, IR break-beams can verify the number of people who enter or exit a room although it is unable to accurately detect the true number of people who currently reside in a particular room and suffers from error accumulation without a method for correcting any discrepancies developed as of yet. Cost analysis from AWS also reveals ongoing expenses, particularly the RDS database charges that consume the majority of the operational expenditure. Other data storage methods should have been proposed to explore more cost-effective solutions. The initial objective of creating a cheaper alternative to commercial systems still holds true when comparing upfront hardware costs as well as monthly subscriptions. In total, approximately £35 was spent on components for the lighting system and by opting for less expensive data permeance solutions such as storing data in S3 buckets or DynamoDB. This would reduce the monthly cost to approximately £2 a month.

5.2 Technical Challenges

Several significant technical challenges arose during implementation that required architectural pivots and new problem-solving approaches. Initial plans involved implementation of MQTT for real time messaging using the AWS IoT Core service, although memory issues were encountered when attempting this approach. The Arduino MQTT libraries were hardcoded to use DRAM for TLS handshakes and despite the ESP-32 WROVER being equipped with 8MB of PSRAM, the DRAM was not sufficient as there would be approximately 160KB of DRAM available for use after Wi-Fi initialisation. This forced the adoption of HTTP polling where the Esp-32 would instead poll an API instead of receiving push notifications from IoT Core, introducing greater latency but simplifying the structure of the cloud-based systems.

Power supply issues also arose during development as LED lights required more amperes than the initial solution of a USB cable could provide. Thus, the change was made to a dedicated power supply connected to a wall socket that was capable of delivering 5V and 10A of power to the hardware components.

5.3 Learning Outcome

This project provided substantial learning opportunities and allowed me to develop various skills as the project spanned multiple technological fields in a single project. All of the technologies aside from cloud infrastructure and API, were ones that had been used before to achieve some other purpose, but this was the first time attempting to incorporate multiple different disciplines in one project. AWS cloud infrastructure involved creating, configuring, and managing RDS Databases, VPC networking and security and access management. I learned to provision Lambda functions with environment variables and read CloudWatch logs when something inevitably did not work as expected. The ESP-32 platform demonstrated real world constraints and the importance of analysing the limitations of technology and planning around said limitations as well as identifying performance requirements.

Android application development revealed the importance of managing threads and how real-world applications function. It also exposed my limited understanding of front-end UI design and gave me a new appreciation for those who peruse it.

I would say the most valuable experience I gained was integrating the different technologies into a single cohesive system and seeing the different technologies and languages interact with each other to achieve an end result.

5.4 Future Improvements

Future improvements could be made to migrate the RDS database to a system that would reduce the cost of data storage such as DynamoDB or S3 storage solutions. This would offer the same level of functionality while significantly reducing the monthly cost.

Integration with voice assistants such as Amazon Alexa or Google Home's Assistant would improve usability and accessibility. More options for automation can also be introduced such as sunrise and sunset synchronization and location-based activation. These new features would drastically improve the performance of the system as a true home automation system and give users new ways to control their homes.

Porting the Android app over to React Native to allow users on both IOS and Android devices to be able to use the app would significantly improve the accessibility of the system as it will allow users from other platforms to use the same systems. The system can also be expanded to allow users to create their own accounts in order to control their lights as the database is currently set up to allow users to connect to and make changes to any set of lights regardless of if they should have access to them. Finally developing a network configuration interface for the microcontroller and mobile app would make set-up and configuration of the system easier for less technologically inclined users.

6.0 Conclusion

6.1 Conclusion

The project successfully demonstrated the feasibility of creating functional smart lighting devices using commercially available hardware, cloud infrastructure and open-source software. The implemented system achieves core project objectives including mobile device-controlled lighting, cloud synchronization, occupancy-based automation and costs below what is expected from commercially available products.

Technical skills developed through the project include AWS infrastructure configuration with VPC networking, and security management, serverless function implementation, with database integration, RESTful API design following hierarchical resource patterns, embedded systems programming with HTTPS certificate validation, and Android mobile development with asynchronous network operations. Most likely the most valuable experience being connecting various technologies together to form a finished product that communicated using HTTP, JSON, SQL and various programming languages as well as physical electronics.

The system exhibits limitations preventing production deployment without refinement, including occupancy tracking accuracy dependence on user discipline and polling-based synchronization introducing up to 2 second latency. However, these represent the trade off of balancing project complexity against developmental time, not technological limitations. These can be improved at a later stage.

The project validates the feasibility of IoT development for technically proficient individuals willing to invest time and resources into demonstrating that commercial-grade functionality can be achieved through integration of readily available components

and services. This project confirms that modern cloud platforms, embedded development tools, and mobile frameworks have matured sufficiently to enable sophisticated home automation implementations outside corporate development contexts.

References

Jose, A.C. and Malekian, R. (2017) 'Improving Smart Home Security: Integrating Logical Sensing Into Smart Home', *IEEE Sensors Journal*, 17(13), pp. 4269–4286. Available at: https://doi.org/10.1109/JSEN.2017.2705045.

Amazon Web Services (2024) *AWS Lambda Developer Guide*. Available at: https://docs.aws.amazon.com/lambda/ (Accessed: 24 April 2026).

Amazon Web Services (2024) *Amazon RDS User Guide*. Available at: https://docs.aws.amazon.com/rds/ (Accessed: 24 April 2026).

FastLED (2024) *FastLED Library Documentation*. Available at: https://github.com/FastLED/FastLED (Accessed: 24 April 2026).

Google (2024) *Android Developer Documentation*. Available at: https://developer.android.com/ (Accessed: 24 April 2026).

Dhaval2404 (2024) *ColorPicker Library for Android*. GitHub repository. Available at: https://github.com/Dhaval2404/ColorPicker (Accessed: 24 April 2026).

Oracle Corporation (2024) *MySQL 8.0 Reference Manual*. Available at: https://dev.mysql.com/doc/ (Accessed: 24 April 2026).

Espressif Systems (2024) *ESP32 Technical Reference Manual*. Available at: https://www.espressif.com/en/products/socs/esp32 (Accessed: 24 April 2026).

Appendix I

Interim Report

Project Proposal:

Project Title:

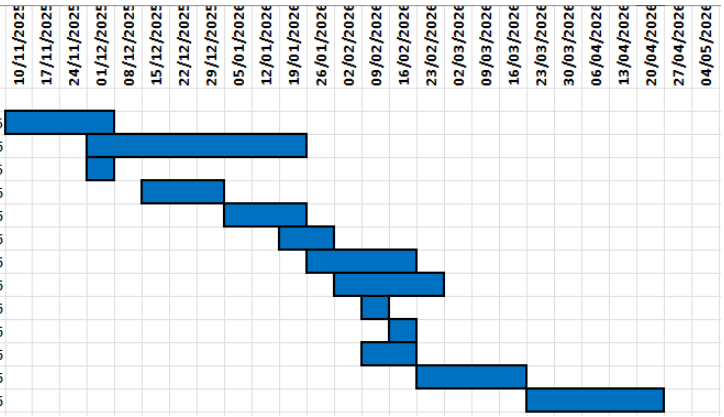
This project aims to investigate and evaluate a control system for a prototype smart home. The prototype will allow for LED lights to be controlled using an android app and light and motion sensors to detect time of day and number of people in a room respectively. Testing will be done using an Arduino that will act as the main controller hub for the LED lights. Testing involve both real events in a physical room and simulated events.

The primary objectives are as follows:

- Determine a range of control systems applicable to smart home automation.
- Select a number of control systems to test
- Implement the testing of said control systems on an embedded device
- Evaluate the results of testing different control systems

Work Plan/Gantt Chart:

Task	Start	End
Example Section 1		
Research	10/11/2025	07/12/2025
Begin 1st Draft Report/Documentation	01/12/2025	27/01/2026
Summarise Findings	01/12/2025	07/12/2025
Design System/Hardware	15/12/2025	05/01/2026
Develop Prototype/Connect Peripherals	05/01/2026	26/01/2026
Design Android Application	19/01/2026	01/02/2026
Write Code for Embedded Controller	25/01/2026	22/02/2026
Write Code for Android Application	01/02/2026	01/03/2026
Test Embedded Controller	09/02/2026	19/02/2026
Test Android Application	14/02/2026	22/02/2026
Testing Analysis	09/02/2026	22/02/2026
Write 2nd Draft Report/Documentation	22/02/2026	26/03/2026
Finalise Report/Documentaion	22/03/2026	26/04/2026



Confirmation of Ethics Approval:

a.r.u. research ethics

05 Nov 2025

Dear Sultan

Principal Investigator: Sultan Olaogun

Research ethics application number: ETH2526-1871

Project Title: Implementation and Performance Analysis of Control Methods in an Embedded Smart Home Prototype

Risk Level: Green (Low)

Acknowledgement of application

Thank you for your ethics application.

I can confirm that, as your research falls into the â€˜greenâ€™ (low risk category), it will not require ethical approval.

You can therefore start your research.

Please note it is your responsibility to ensure that you comply with ARUâ€™s Research Ethics Policy and the Code of Practice for Applying for Ethical Approval, available at www.aru.ac.uk/researchethics

For your information, research falling into the green category is subject to audit.

Good luck with your research.

Yours sincerely,

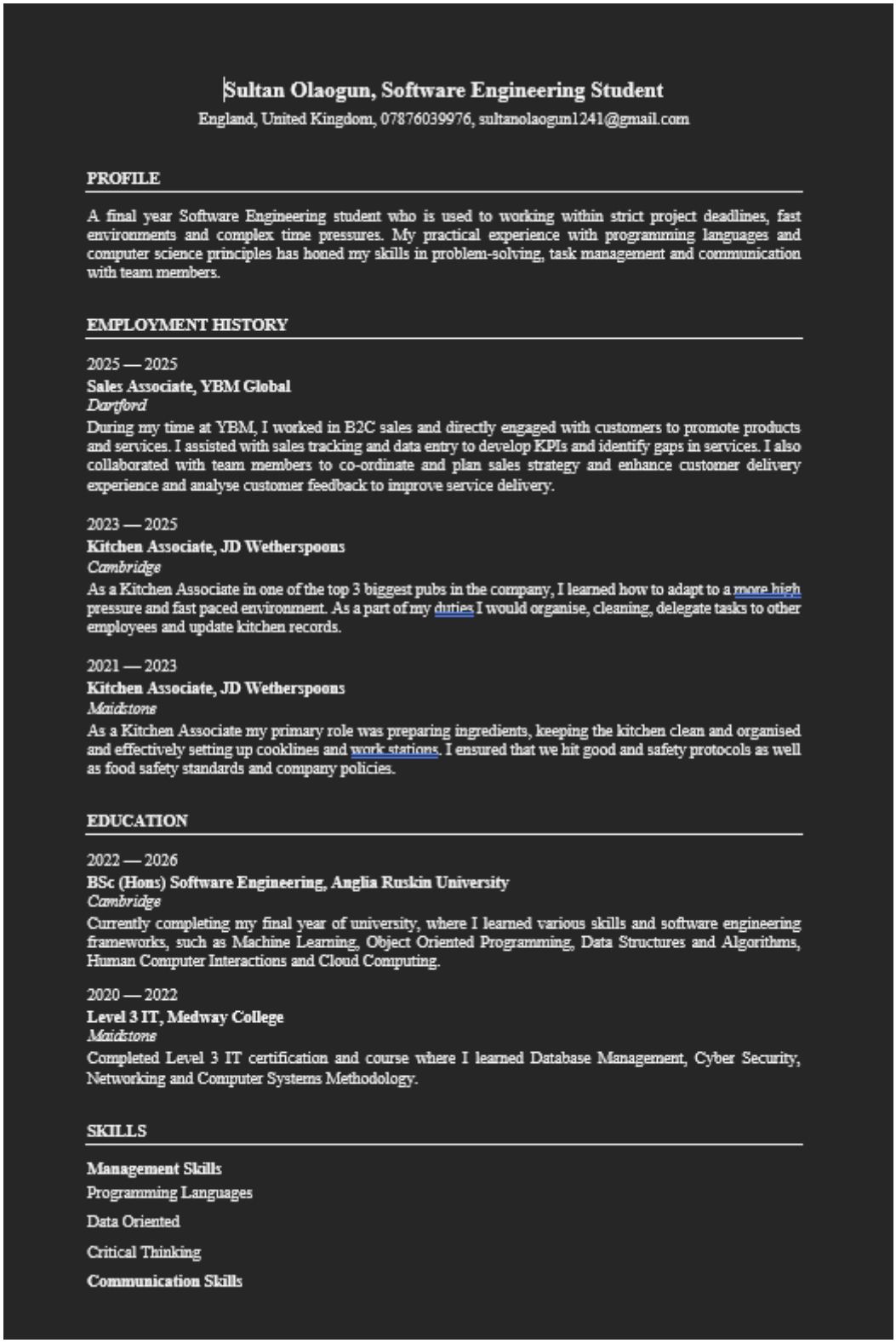
Philip Payne

ARU Chelmsford, Bishop Hall Lane, Chelmsford, CM1 1SQ 01245 493 131

ARU Cambridge, East Road, Cambridge, CB1 1PT 01245 493 131

Ethics ETH2526-1871 : Mr Sultan Olaogun (Low risk: Green) - Implementation and Performance Analysis of Control Methods in an Embedded Smart Home Prototype

CV & Exit Plan:



Sultan Olaogun, Software Engineering Student

England, United Kingdom, 07876039976, sultanolaogun1241@gmail.com

PROFILE

A final year Software Engineering student who is used to working within strict project deadlines, fast environments and complex time pressures. My practical experience with programming languages and computer science principles has honed my skills in problem-solving, task management and communication with team members.

EMPLOYMENT HISTORY

2025 — 2025
Sales Associate, YBM Global
Dartford
During my time at YBM, I worked in B2C sales and directly engaged with customers to promote products and services. I assisted with sales tracking and data entry to develop KPIs and identify gaps in services. I also collaborated with team members to co-ordinate and plan sales strategy and enhance customer delivery experience and analyse customer feedback to improve service delivery.

2023 — 2025
Kitchen Associate, JD Wetherspoons
Cambridge
As a Kitchen Associate in one of the top 3 biggest pubs in the company, I learned how to adapt to a more high pressure and fast paced environment. As a part of my duties I would organise, cleaning, delegate tasks to other employees and update kitchen records.

2021 — 2023
Kitchen Associate, JD Wetherspoons
Maidstone
As a Kitchen Associate my primary role was preparing ingredients, keeping the kitchen clean and organised and effectively setting up cooklines and work stations. I ensured that we hit good and safety protocols as well as food safety standards and company policies.

EDUCATION

2022 — 2026
BSc (Hons) Software Engineering, Anglia Ruskin University
Cambridge
Currently completing my final year of university, where I learned various skills and software engineering frameworks, such as Machine Learning, Object Oriented Programming, Data Structures and Algorithms, Human Computer Interactions and Cloud Computing.

2020 — 2022
Level 3 IT, Medway College
Maidstone
Completed Level 3 IT certification and course where I learned Database Management, Cyber Security, Networking and Computer Systems Methodology.

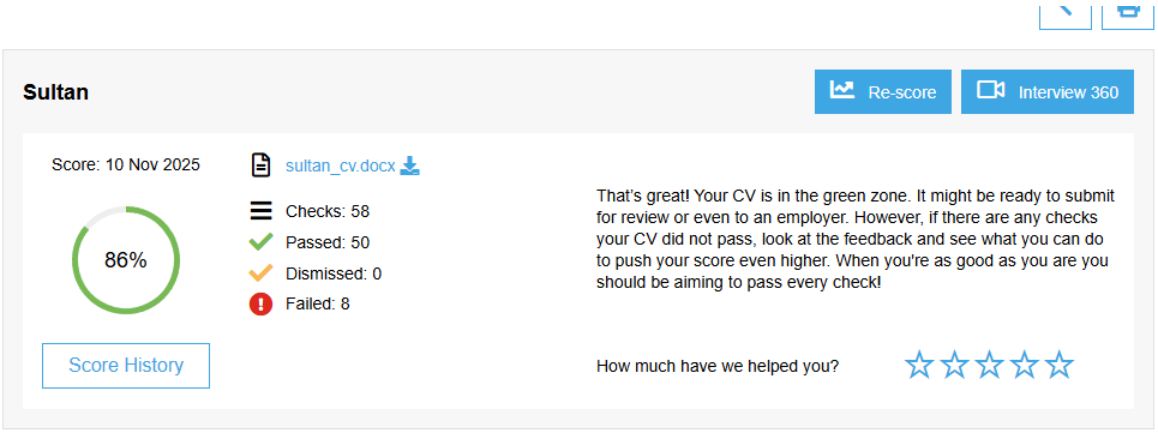
SKILLS

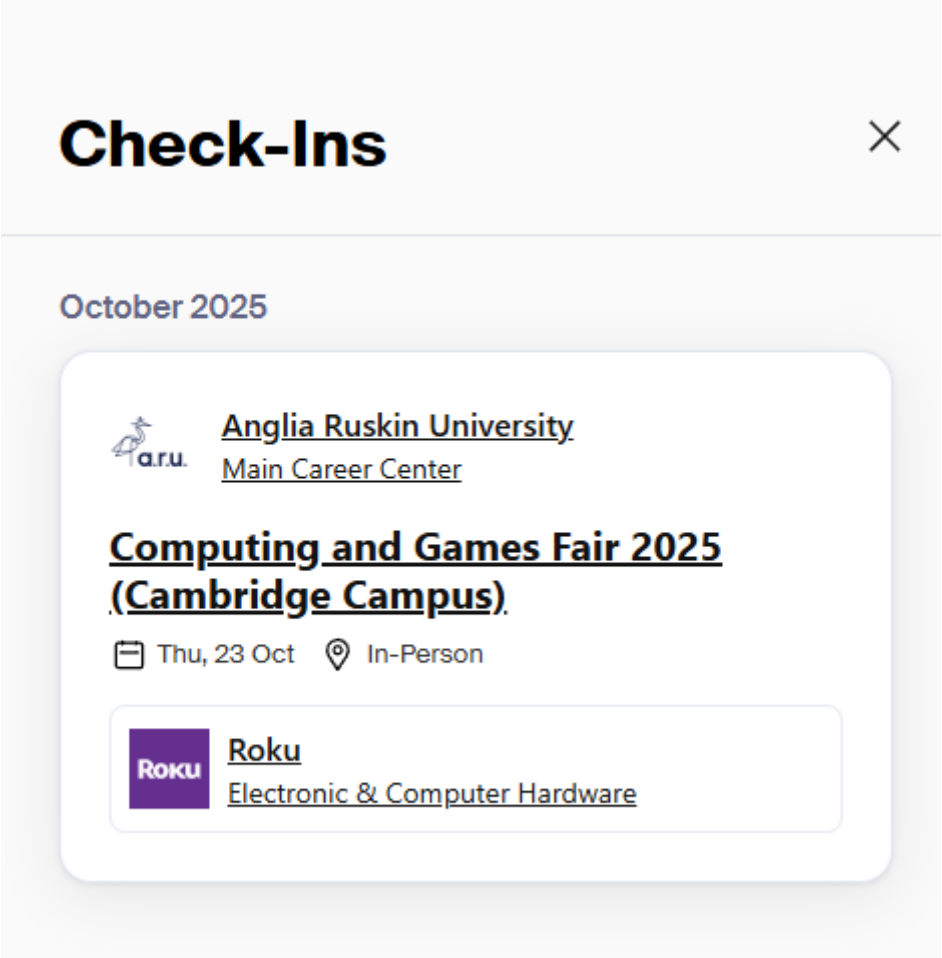
Management Skills
Programming Languages
Data Oriented
Critical Thinking
Communication Skills

After completing university, I plan to improve my employability by working on various projects and continuing to learn new skills. I aim to seek a graduate role in a software engineering related field where I will

be able to expand my skills and gain valuable experience working with industry experts.

Employability Activities:





Appendix II

LED Light Controller System

Android - ESP32 - REST API - Cloud Database

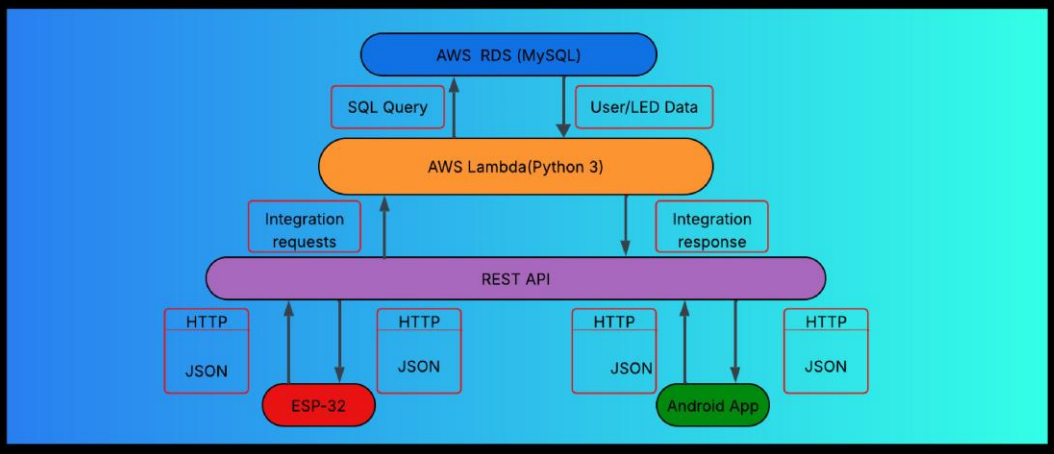
Introduction

This LED Light Controller System project enables the user to set up and control LED lights from an android device.

The system acts as a bridge for communication from the users device to an ESP-32 which is physically connected to an individually addressable light strip.

Project Aims

- Build and design cross platform LED Controller
- Implement API using AWS API Gateway
- Store and retrieve data using AWS Cloud Database
- Build and Android app for real-time user input



Methodology

Android App built in Java using android studio, allows users to control lights with UI

ESP-32 programmed using Arduino IDE and C++. Receives data from API to drive LED lights

API acts as middle-ware while Lambda processes requests

Key Features

- Real time updates/control
- Multiple "room" management
- Multi user support/Individual Preferences
- Persistent Cloud storage of user settings
- Low-latency delivery

Technology Stack

Android/Java | ESP-32/C++ | REST API | MySQL | Lambda/Python

Sultan Olaogun

SSID:2228531 Email: Sultanolaogun124@gmail.com

Appendix III

API Gateway Structure